

PROTOTYPE HOUSE

Patrón de Diseño: *Prototype*

Clonación de prototipos para construir una Casa compuesta por figuras geométricas

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Agosto 2026

Índice

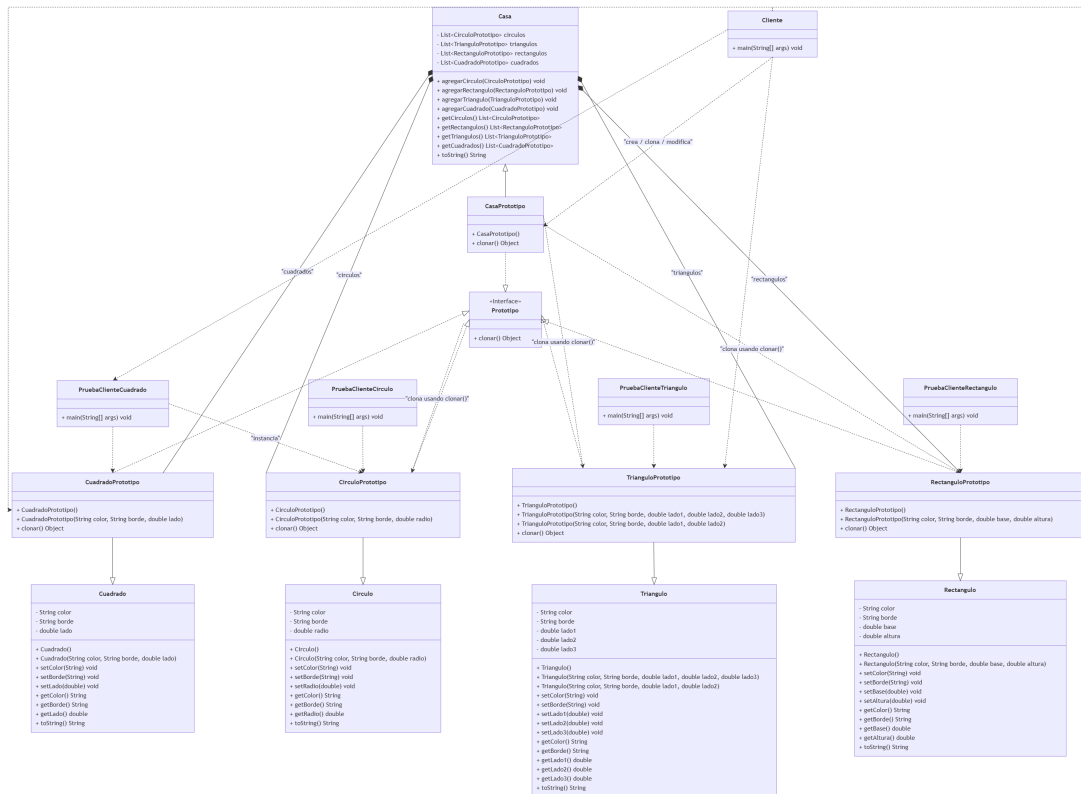
1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Clase Casa	3
2.2. Clases de Figuras	6
2.2.1. Circulo.java	6
2.2.2. Cuadrado.java	7
2.2.3. Rectangulo.java	8
2.2.4. Triangulo.java	10
2.3. Prototipos	12
2.3.1. Casa	12
2.3.2. Rectangulo	13
2.3.3. Cuadrado	14
2.3.4. Circulo	15
2.3.5. Triangulo	15
3. Código de Cliente	17
3.1. Cliente	17
4. Pruebas	21
5. Código y Enlaces	24
5.1. Repositorio GitHub	24
5.2. Estructura de Paquetes	24

Diagrama UML

Descripción General

El proyecto PrototypeHouse implementa el patrón de diseño **Prototype** para clonar objetos que representan componentes de una casa (Techo, Paredes, Ventanas, Puerta) usando prototipos de figuras geométricas. El objetivo es permitir la copia (clonación) de estructuras complejas (Casa) y luego modificar la copia sin alterar el original.

Diagrama de Clases



Explicación de las Clases

Clase Casa

Descripción: Clase contenedora que mantiene listas de prototipos para cada tipo de figura y ofrece getters/setters y métodos para agregar elementos.

```
1 package prototypehouse;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import prototypehouse.Prototipos.CirculoPrototipo;
6 import prototypehouse.Prototipos.CuadradoPrototipo;
7 import prototypehouse.Prototipos.RectanguloPrototipo;
8 import prototypehouse.Prototipos.TrianguloPrototipo;
9
10 /**
11  *
12  * @author natha
13  */
14 public class Casa {
15
16     private List<CirculoPrototipo> circulos = new ArrayList<>();
17     private List<RectanguloPrototipo> rectangulos = new ArrayList
18         <>();
19     private List<TrianguloPrototipo> triangulos = new ArrayList
20         <>();
21     private List<CuadradoPrototipo> cuadrados = new ArrayList<>()
22         ;
23
24     //Constructores
25
26     public Casa() {
27     }
28
29     public void agregarCirculo(CirculoPrototipo c) {
30         this.circulos.add(c);
31     }
32
33     public void agregarRectangulo(RectanguloPrototipo r) {
34         this.rectangulos.add(r);
35     }
36
37     public void agregarTriangulo(TrianguloPrototipo t) {
38         this.triangulos.add(t);
39     }
40
41     public void agregarCuadrado(CuadradoPrototipo cdr) {
42         this.cuadrados.add(cdr);
43     }
44 }
```

```
41
42 //Setters
43
44 public void setCirculos(List<CirculoPrototipo> circulos) {
45     this.circulos = circulos;
46 }
47
48 public void setRectangulos(List<RectanguloPrototipo>
49     rectangulos) {
50     this.rectangulos = rectangulos;
51 }
52
53 public void setTriangulos(List<TrianguloPrototipo> triangulos
54     ) {
55     this.triangulos = triangulos;
56 }
57
58 public void setCuadrados(List<CuadradoPrototipo> cuadrados)
59     {
60     this.cuadrados = cuadrados;
61 }
62
63 //Getters
64
65 public List<CirculoPrototipo> getCirculos() {
66     return circulos;
67 }
68
69 public List<RectanguloPrototipo> getRectangulos() {
70     return rectangulos;
71 }
72
73 public List<TrianguloPrototipo> getTriangulos() {
74     return triangulos;
75 }
76
77 public List<CuadradoPrototipo> getCuadrados() {
78     return cuadrados;
79 }
80
81 //to string
82
83 @Override
84 public String toString() {
85
86     return "\n" +
87         "└─Techo─┐" + this.triangulos + "\n" +
88         "└─Paredes:─┐" + this.rectangulos + "\n" +
```

```
89         "└─Puerta:└" + this.cuadrados + "\n" +  
90         "└─Ventanas└:└" + this.circulos + "\n" ;  
91     }  
92  
93  
94 }
```

Clases de Figuras

Las siguientes clases representan las figuras con sus atributos y métodos accesoros. Se usan como contenido de los prototipos y para imprimir/describir componentes.

2.2.1. Circulo.java

```
1 package prototypehouse;
2
3 /**
4  *
5  * @author natha
6  */
7 public class Circulo {
8
9     private String color;
10    private String borde;
11    private double radio;
12
13    public Circulo(String color, String borde, double radio) {
14        this.color = color;
15        this.borde = borde;
16        this.radio = radio;
17    }
18
19    public Circulo() {
20    }
21
22    //Setters
23
24    public void setColor(String color) {
25        this.color = color;
26    }
27
28    public void setBorde(String borde) {
29        this.borde = borde;
30    }
31
32    public void setRadio(double radio) {
33        this.radio = radio;
34    }
35
36    //Getters
37
38    public String getColor() {
39        return color;
40    }
41
42    public String getBorde() {
43        return borde;
44    }
```

```
45
46     public double getRadio() {
47         return radio;
48     }
49
50     //to String
51     @Override
52     public String toString() {
53         return "Circulo{" + "color=" + color + ",\nborde=" + borde
54             + ",\nradio=" + radio + '}';
55     }
56
57
58
59 }
```

2.2.2. Cuadrado.java

```
1  package prototypehouse;
2
3  /**
4   *
5   * @author natha
6   */
7  public class Cuadrado {
8
9      private String color;
10     private String borde;
11     private double lado;
12
13     //Constructores
14
15     public Cuadrado() {
16     }
17
18     public Cuadrado(String color, String borde, double lado) {
19         this.color = color;
20         this.borde = borde;
21         this.lado = lado;
22     }
23
24     //Setters
25
26     public void setColor(String color) {
27         this.color = color;
28     }
29
30     public void setBorde(String borde) {
31         this.borde = borde;
32     }
```

```
33     }
34
35     public void setLado(double lado) {
36         this.lado = lado;
37     }
38
39     //Getters
40
41     public String getColor() {
42         return color;
43     }
44
45     public String getBorde() {
46         return borde;
47     }
48
49     public double getLado() {
50         return lado;
51     }
52
53     //to String
54
55     @Override
56     public String toString() {
57         return "Cuadrado{" + "color=" + color + ",┐borde=" +
58             borde + ",┐lado=" + lado + '}';
59     }
60 }
```

2.2.3. Rectangulo.java

```
1 package prototypehouse;
2
3 /**
4  *
5  * @author natha
6  */
7 public class Rectangulo {
8
9     private String color;
10    private String borde;
11    private double base;
12    private double altura;
13
14    //Constructores
15
16    public Rectangulo() {
17    }
18
19 }
```

```
20     public Rectangulo(String color, String borde, double base,
21         double altura) {
22         this.color = color;
23         this.borde = borde;
24         this.base = base;
25         this.altura = altura;
26     }
27
28     //Setters
29
30     public void setColor(String color) {
31         this.color = color;
32     }
33
34     public void setBorde(String borde) {
35         this.borde = borde;
36     }
37
38     public void setBase(double base) {
39         this.base = base;
40     }
41
42     public void setAltura(double altura) {
43         this.altura = altura;
44     }
45
46     //Getters
47
48     public String getColor() {
49         return color;
50     }
51
52     public String getBorde() {
53         return borde;
54     }
55
56     public double getBase() {
57         return base;
58     }
59
60     public double getAltura() {
61         return altura;
62     }
63
64     //to String
65
66     @Override
67     public String toString() {
68         return "Rectangulo{" + "color=" + color + ",_borde=" +
69             borde + ",_base=" + base + ",_altura=" + altura + '}';
70     }
```

69
70 }

2.2.4. Triangulo.java

```
1 package prototypehouse;
2
3 /**
4  *
5  * @author natha
6  */
7 public class Triangulo {
8
9     private String color;
10    private String borde;
11    private double lado1;
12    private double lado2;
13    private double lado3;
14
15    //Constructores
16
17    public Triangulo() {
18    }
19
20
21
22    public Triangulo(String color, String borde, double lado1,
23        double lado2, double lado3) {
24        this.color = color;
25        this.borde = borde;
26        this.lado1 = lado1;
27        this.lado2 = lado2;
28        this.lado3 = lado3;
29    }
30
31    public Triangulo(String color, String borde, double lado1,
32        double lado2) {
33        this.color = color;
34        this.borde = borde;
35        this.lado1 = lado1;
36        this.lado2 = lado2;
37        this.lado3 = Math.sqrt((lado1*lado1)+(lado2*lado2));
38    }
39
40    //Setters
41
42    public void setColor(String color) {
43        this.color = color;
44    }
```

```
45
46     public void setBorde(String borde) {
47         this.borde = borde;
48     }
49
50     public void setLado1(double lado1) {
51         this.lado1 = lado1;
52     }
53
54     public void setLado2(double lado2) {
55         this.lado2 = lado2;
56     }
57
58     public void setLado3(double lado3) {
59         this.lado3 = lado3;
60     }
61
62     //Getters
63
64     public String getColor() {
65         return color;
66     }
67
68     public String getBorde() {
69         return borde;
70     }
71
72     public double getLado1() {
73         return lado1;
74     }
75
76     public double getLado2() {
77         return lado2;
78     }
79
80     public double getLado3() {
81         return lado3;
82     }
83
84     //to String
85
86     @Override
87     public String toString() {
88         return "Triangulo{" + "color=" + color + ",_borde=" +
            borde + ",_lado1=" + lado1 + ",_lado2=" + lado2 + ",_
            lado3=" + lado3 + '}';
89     }
90
91 }
```

Prototipos

Descripción general

Se implementa el patrón Prototype por medio de la interfaz `Prototipo` (contrato con el método `clonar()`) y las clases prototipo concretas que extienden las clases de datos (`Circulo`, `Rectangulo`, `Triangulo`, `Cuadrado`) añadiendo la capacidad de clonar sus atributos. Además existe el prototipo compuesto `CasaPrototipo` que clona una casa completa delegando la clonación a sus prototipos hijos. La clonación es manual: cada prototipo crea una nueva instancia y copia sus atributos relevantes (color, borde, dimensiones) para obtener copias independientes.

Interfaz Prototipo

```
1 package prototypehouse.Prototipos;
2
3 /**
4  *
5  * @author natha
6  */
7 public interface Prototipo {
8
9     public Object clonar();
10
11 }
```

2.3.1. Casa

Explicación: `CasaPrototipo` extiende `Casa` y implementa `Prototipo`. Su método `clonar()` crea una nueva instancia de `CasaPrototipo` y clona recursivamente cada prototipo contenido (círculos, rectángulos, triángulos) delegando en el método `clonar()` de cada uno.

```
1 package prototypehouse.Prototipos;
2
3 import prototypehouse.Casa;
4
5 /**
6  *
7  * @author natha
8  */
9 public class CasaPrototipo extends Casa implements Prototipo{
10
11
12     //Constructor
13
14
15     public CasaPrototipo() {
16         super();
17     }
18
19     @Override
20     public Object clonar() {
```

```
21     CasaPrototipo casaClon = new CasaPrototipo();
22
23     for (CirculoPrototipo c : this.getCirculos()) {
24         casaClon.agregarCirculo((CirculoPrototipo) c.clonar()
25             );
26     }
27
28     for (RectanguloPrototipo r : this.getRectangulos()) {
29         casaClon.agregarRectangulo((RectanguloPrototipo) r.
30             clonar());
31     }
32
33     for (TrianguloPrototipo t : this.getTriangulos()) {
34         casaClon.agregarTriangulo((TrianguloPrototipo) t.
35             clonar());
36     }
37
38     return casaClon;
39 }
```

2.3.2. Rectangulo

Explicación: RectanguloPrototipo extiende Rectangulo y implementa Prototipo. Su clonar() crea una nueva instancia y copia los atributos color, borde, base y altura.

```
1 package prototypehouse.Prototipos;
2
3 import prototypehouse.Rectangulo;
4
5 /**
6  *
7  * @author natha
8  */
9 public class RectanguloPrototipo extends Rectangulo implements
10     Prototipo{
11
12     //Constructor
13
14     public RectanguloPrototipo() {
15         super();
16     }
17
18     public RectanguloPrototipo(String color, String borde, double
19         base, double altura) {
20         super(color, borde, base, altura);
21     }
22
23     @Override
```

```
22     public Object clonar() {
23         RectanguloPrototipo recClon = new RectanguloPrototipo();
24         recClon.setColor(this.getColor());
25         recClon.setBorde(this.getBorde());
26         recClon.setBase(this.getBase());
27         recClon.setAltura(this.getAltura());
28         return recClon;
29     }
30
31
32 }
```

2.3.3. Cuadrado

Explicación: CuadradoPrototipo extiende Cuadrado y copia manualmente color, borde y lado en su método clonar().

```
1 package prototypehouse.Prototipos;
2
3 import prototypehouse.Cuadrado;
4
5 /**
6  *
7  * @author natha
8  */
9 public class CuadradoPrototipo extends Cuadrado implements
10     Prototipo{
11
12     //Constructores
13
14     public CuadradoPrototipo() {
15         super();
16     }
17
18     public CuadradoPrototipo(String color, String borde, double
19         lado) {
20         super(color, borde, lado);
21     }
22
23     @Override
24     public Object clonar() {
25
26         CuadradoPrototipo cuadClon = new CuadradoPrototipo();
27         cuadClon.setColor(this.getColor());
28         cuadClon.setBorde(this.getBorde());
29         cuadClon.setLado(this.getLado());
30         return cuadClon;
31     }
32 }
```

2.3.4. Circulo

Explicación: CirculoPrototipo extiende Circulo y en clonar() crea una nueva instancia copiando color, borde y radio.

```
1 package prototypehouse.Prototipos;
2
3 import prototypehouse.Circulo;
4
5 /**
6  *
7  * @author natha
8  */
9 public class CirculoPrototipo extends Circulo implements
    Prototipo{
10
11     //Constructores
12
13     public CirculoPrototipo(String color, String borde, double
        radio) {
14         super(color, borde, radio);
15     }
16
17     public CirculoPrototipo() {
18         super();
19     }
20
21
22     @Override
23     public Object clonar() {
24         CirculoPrototipo circuloClon = new CirculoPrototipo();
25         circuloClon.setColor(this.getColor());
26         circuloClon.setBorde(this.getBorde());
27         circuloClon.setRadio(this.getRadio());
28         return circuloClon;
29     }
30
31 }
```

2.3.5. Triangulo

Explicación: TrianguloPrototipo extiende Triangulo y su clonar() copia color, borde y las tres dimensiones (lado1, lado2, lado3). También provee constructores útiles para triángulo con 2 o 3 lados.

```
1 package prototypehouse.Prototipos;
2
3 import prototypehouse.Triangulo;
4
5 /**
6  *
7  * @author natha
```

```
8  */
9  public class TrianguloPrototipo extends Triangulo implements
    Prototipo{
10
11
12     public TrianguloPrototipo() {
13         super();
14     }
15
16     public TrianguloPrototipo(String color, String borde, double
        lado1, double lado2, double lado3) {
17         super(color, borde, lado1, lado2, lado3);
18     }
19
20     //Constructor
21     public TrianguloPrototipo(String color, String borde, double
        lado1, double lado2) {
22         super(color, borde, lado1, lado2);
23     }
24
25     @Override
26     public Object clonar() {
27         TrianguloPrototipo triangClon = new TrianguloPrototipo();
28         triangClon.setColor(this.getColor());
29         triangClon.setBorde(this.getBorde());
30         triangClon.setLado1(this.getLado1());
31         triangClon.setLado2(this.getLado2());
32         triangClon.setLado3(this.getLado3());
33         return triangClon;
34     }
35
36
37
38 }
```

Código de Cliente

Cliente

Descripción: Clase que interactúa con el usuario, solicita parámetros para construir prototipos, crea una CasaPrototipo, la clona y permite modificar la copia.

```
1 package prototypehouse;
2
3 import java.util.Scanner;
4 import prototypehouse.Prototipos.CasaPrototipo;
5 import prototypehouse.Prototipos.CirculoPrototipo;
6 import prototypehouse.Prototipos.CuadradoPrototipo;
7 import prototypehouse.Prototipos.RectanguloPrototipo;
8 import prototypehouse.Prototipos.TrianguloPrototipo;
9
10 /**
11  *
12  * @author natha
13  */
14 public class Cliente {
15
16     public static void main( String[] args){
17
18         //Techo
19
20         Scanner sc = new Scanner(System.in);
21         System.out.println("Configuracion del Techo (Triangulo): "
22             );
23         System.out.print("Ingrese el color:");
24         String colorTecho = sc.nextLine();
25         System.out.print("Ingrese borde:");
26         String bordeTecho = sc.nextLine();
27         System.out.print("Ingrese el lado 1:");
28         double lado1 = sc.nextDouble();
29         System.out.print("Ingrese el lado 2:");
30         double lado2 = sc.nextDouble();
31         System.out.print("Ingrese el lado 3:");
32         double lado3 = sc.nextDouble();
33         sc.nextLine();
34
35         TrianguloPrototipo techo = new TrianguloPrototipo(
36             colorTecho, bordeTecho, lado1, lado2, lado3);
37
38         // Paredes
39
40         System.out.println("\nConfiguracion Pared (Rectangulo)");
41         ;
42         System.out.print("Ingrese el color:");
43         String colorPared = sc.nextLine();
44         System.out.print("Ingrese el borde:");
45         String bordePared = sc.nextLine();
```

```
43      System.out.print("Ingrese la base: ");
44      double basePared = sc.nextDouble();
45      System.out.print("Ingrese la altura: ");
46      double alturaPared = sc.nextDouble();
47      sc.nextLine();
48
49      RectanguloPrototipo pared = new RectanguloPrototipo(
50          colorPared, bordePared, basePared, alturaPared);
51
52      //Ventana
53
54      System.out.println("\nConfiguracion Ventana (Circulo)");
55      System.out.print("Ingrese el color: ");
56      String colorVentana = sc.nextLine();
57      System.out.print("Ingrese el borde: ");
58      String bordeVentana = sc.nextLine();
59      System.out.print("Ingrese el radio: ");
60      double radioVentana = sc.nextDouble();
61      sc.nextLine();
62
63      CirculoPrototipo ventana = new CirculoPrototipo(
64          colorVentana, bordeVentana, radioVentana);
65
66      // Puerta
67
68      System.out.println("\nConfiguracion Puerta (Cuadrado)");
69      System.out.print("Ingrese el color: ");
70      String colorPuerta = sc.nextLine();
71      System.out.print("Ingrese el borde: ");
72      String bordePuerta = sc.nextLine();
73      System.out.print("Ingrese el lado: ");
74      double ladoPuerta = sc.nextDouble();
75      sc.nextLine();
76
77      CuadradoPrototipo puerta = new CuadradoPrototipo(
78          colorPuerta, bordePuerta, ladoPuerta);
79
80
81      CasaPrototipo casa = new CasaPrototipo();
82      casa.agregarCirculo(ventana);
83      casa.agregarRectangulo(pared);
84      casa.agregarTriangulo(techo);
85      casa.agregarCuadrado(puerta);
86
87      System.out.println("\nCasa original: ");
88      System.out.println(casa.toString());
89
90      CasaPrototipo casaClon = (CasaPrototipo) casa.clonar();
91      System.out.println("Casa clonada: ");
92      System.out.println(casaClon.toString());
93
94      //Modificaciones
```

```
91
92     int opcion = 0;
93
94     do {
95
96         System.out.println("\n--_MENU_Modificacion_Clon_--");
97         System.out.println("1._Modificar_Techo_(Color_y_dimencion)");
98         System.out.println("2._Modificar_Pared_(Color_y_dimencion)");
99         System.out.println("3._Modificar_Ventana_(Color_y_dimencion)");
100        System.out.println("4._Salir");
101        System.out.print("Selecione una opcion:");
102
103        if (sc.hasNextInt()) {
104            opcion = sc.nextInt();
105            sc.nextLine();
106
107            switch (opcion) {
108                case 1:
109
110                    System.out.print("Nuevo_color:");
111
112                    casaClon.getTriangulos().get(0).setColor(sc.nextLine());
113                    System.out.print("Nuevo_valor_de_la_base:");
114                    casaClon.getTriangulos().get(0).setLado1(sc.nextDouble());
115                    sc.nextLine();
116
117                    System.out.println("\nCasa_original:");
118                    System.out.println(casa.toString());
119                    System.out.println("Casa_clonada:");
120                    System.out.println(casaClon.toString());
121
122                    break;
123
124                case 2:
125
126                    System.out.print("Nuevo_color:");
127                    casaClon.getRectangulos().get(0).setColor(sc.nextLine());
128                    System.out.print("Nueva_altura:");
129                    casaClon.getRectangulos().get(0).setAltura(sc.nextDouble());
130                    sc.nextLine();
131
132                    System.out.println("\nCasa_original:");
```

```
133         System.out.println(casa.toString());
134         System.out.println("Casa clonada:");
135         System.out.println(casaClon.toString());
136
137         break;
138
139         case 3:
140
141             System.out.print("Nuevo color:");
142             casaClon.getCirculos().get(0).setColor(sc
143                 .nextLine());
144             System.out.print("Nuevo radio:");
145             casaClon.getCirculos().get(0).setRadio(sc
146                 .nextDouble());
147             sc.nextLine();
148
149             System.out.println("\nCasa original:");
150             System.out.println(casa.toString());
151             System.out.println("Casa clonada:");
152             System.out.println(casaClon.toString());
153
154             break;
155
156         case 4:
157             System.out.println("\nSaliendo..");
158             break;
159
160         default:
161             System.out.println("\nError.");
162     }
163 } else {
164     System.out.println("\nError, ingrese una opcion
165     valida.");
166     sc.next();
167 }
168 } while (opcion != 4);
169
170 sc.close();
171 }
```

Compilacion:

```

Output - PrototypeHouse (run) x
run:
Configuracion el Techo (Triangulo) :
Ingrese el color: Verde
Ingrese borde: Negro
Ingrese el lado 1: 3
Ingrese el lado 2: 2
Ingrese el lado 3: 5

Configuracion Pared (Rectangulo)
Ingrese el color: Naranja
Ingrese el borde: Negro
Ingrese la base: 1
Ingrese la altura: 1

Configuracion Ventana (Circulo)
Ingrese el color: Azul
Ingrese el borde: 2
Ingrese el radio: 2

Configuracion Puerta (Cuadrado)
Ingrese el color: Rojo
Ingrese el borde: Negro
Ingrese el lado: 1

Casa original:

Techo : [Triangulo(color=Verde, borde=Negro, lado1=3.0, lado2=2.0, lado3=5.0)]
- Paredes: [Rectangulo(color=Naranja, borde=Negro, base=1.0, altura=1.0)]
- Puerta: [Cuadrado(color=Rojo, borde=Negro, lado=1.0)]
- Ventanas : [Circulo(color=Azul, borde=2, radio=2.0)]

Casa clonada:

Techo : [Triangulo(color=Verde, borde=Negro, lado1=3.0, lado2=2.0, lado3=5.0)]
- Paredes: [Rectangulo(color=Naranja, borde=Negro, base=1.0, altura=1.0)]
- Puerta: []
- Ventanas : [Circulo(color=Azul, borde=2, radio=2.0)]

-- MENU Modificacion Clon --
1. Modificar Techo (Color y dimension)
2. Modificar Pared (Color y dimension)
3. Modificar Ventana (Color y dimension)
4. Salir
Seleccione una opcion: 1
Nuevo color: Morado
Nuevo valor de la base: 2

Casa original:

Techo : [Triangulo(color=Verde, borde=Negro, lado1=3.0, lado2=2.0, lado3=5.0)]
- Paredes: [Rectangulo(color=Naranja, borde=Negro, base=1.0, altura=1.0)]
- Puerta: [Cuadrado(color=Rojo, borde=Negro, lado=1.0)]
- Ventanas : [Circulo(color=Azul, borde=2, radio=2.0)]

Casa clonada:

Techo : [Triangulo(color=Morado, borde=Negro, lado1=2.0, lado2=2.0, lado3=5.0)]
- Paredes: [Rectangulo(color=Naranja, borde=Negro, base=1.0, altura=1.0)]
- Puerta: []
- Ventanas : [Circulo(color=Azul, borde=2, radio=2.0)]

```

Figura 1: Captura de ejecución del cliente.

Pruebas

Descripción: A continuación se presenta el código de prueba utilizado para instanciar y clonar el prototipo del rectángulo como ejemplo base.

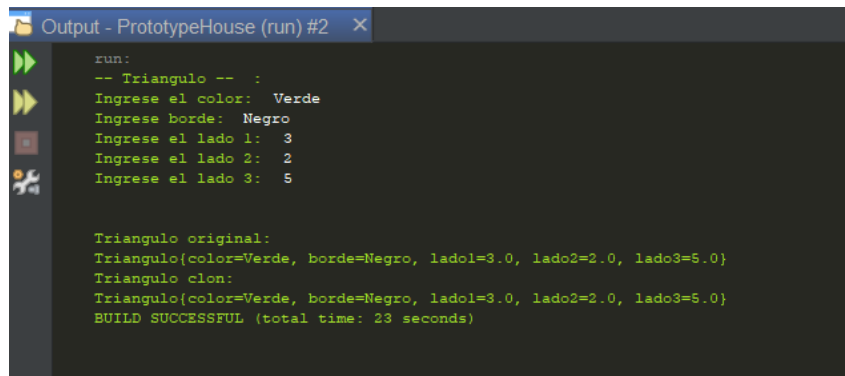
```

1
2 package prototypehouse.Pruebas;
3
4 import java.util.Scanner;
5 import prototypehouse.Prototipos.RectanguloPrototipo;
6
7 /**
8  *
9  * @author natha
10  */
11 public class PruebaClienteRectangulo {
12
13     public static void main( String[] args){

```

```
14
15     Scanner sc = new Scanner(System.in);
16     System.out.println("--_Rectangulo_--_:_");
17     System.out.print("Ingrese_el_color:_");
18     String color = sc.nextLine();
19     System.out.print("Ingrese_borde:_");
20     String borde = sc.nextLine();
21     System.out.print("Ingrese_la_base:_");
22     double base = sc.nextDouble();
23     System.out.print("Ingrese_la_altura:_");
24     double altura = sc.nextDouble();
25     sc.nextLine();
26
27     RectanguloPrototipo rect = new RectanguloPrototipo(color,
28         borde, base, altura);
29
30     RectanguloPrototipo rectClon = (RectanguloPrototipo) rect
31         .clonar();
32
33     System.out.println("\n");
34     System.out.println("Rectangulo_original:_");
35     System.out.println(rect.toString());
36
37     System.out.println("Rectangulo_clon:_");
38     System.out.println(rectClon.toString());
39
40     sc.close();
    }
```

Triángulo



```
Output - PrototypeHouse (run) #2 x
run:
-- Triangulo -- :
Ingrese el color: Verde
Ingrese borde: Negro
Ingrese el lado 1: 3
Ingrese el lado 2: 2
Ingrese el lado 3: 5

Triangulo original:
Triangulo(color=Verde, borde=Negro, lado1=3.0, lado2=2.0, lado3=5.0)
Triangulo clon:
Triangulo(color=Verde, borde=Negro, lado1=3.0, lado2=2.0, lado3=5.0)
BUILD SUCCESSFUL (total time: 23 seconds)
```

Figura 2: Captura de ejecución de las pruebas del Triángulo.

Código y Enlaces

Repositorio GitHub

- URL del proyecto: <https://github.com/Thalyhx/PrototypeHouse>

Estructura de Paquetes

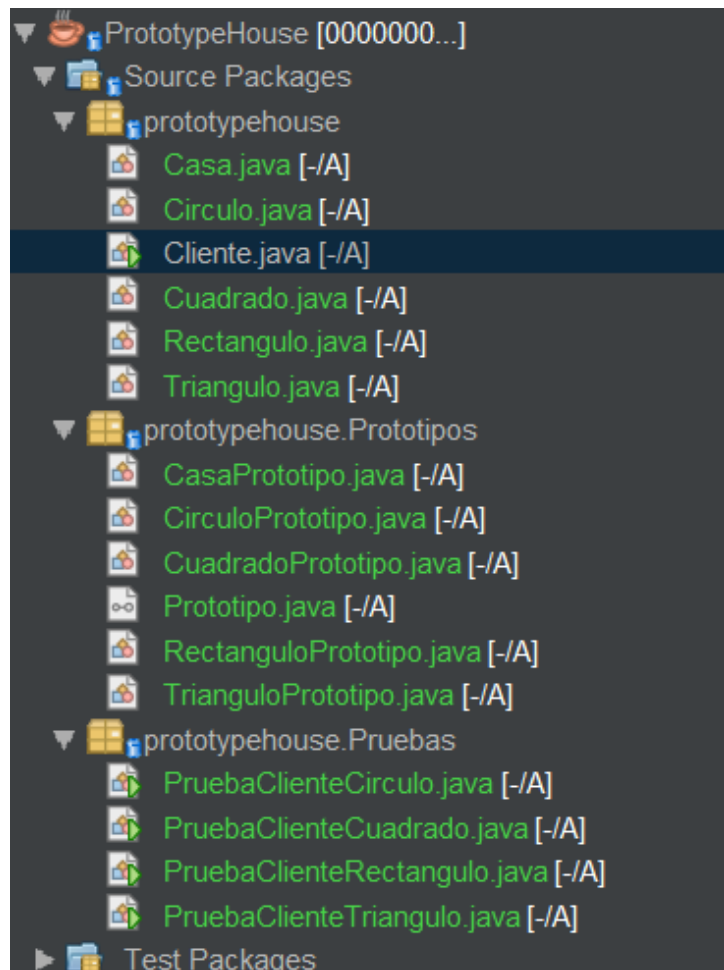


Figura 3: Paquetes